

MÁSTER EN BIOINFORMÁTICA Y BIOESTADÍSTICA

PEC 3

Machine Learning, Shiny y Bioinformática con R

Software para el Análisis de Datos

Autora: Dra. Marta Barea Sepúlveda

Fecha: 20 de mayo de 2026

Índice

1. Introducción	1
2. Sección 1: Modelos de clasificación y validación	2
2.1. Ejercicio 1. Clasificación de Tumores Mamarios con Breast Cancer Wisconsin	2
2.1.1. Enunciado	2
2.1.2. Resolución	3
2.2. Ejercicio 2. Humedad del suelo y cobertura vegetal	17
2.2.1. Enunciado	17
2.2.2. Resolución	18
3. Sección 2: Aplicaciones con Shiny y ética	23
3.1. Ejercicio 3. Aplicación interactiva con Shiny	23
3.1.1. Enunciado	23
3.1.2. Resolución	24
3.2. Ejercicio 4. Ética en aplicaciones Shiny	33
3.2.1. Enunciado	33
3.2.2. Resolución	33
4. Sección 3: Bioconductor y análisis genómico	34
4.1. Ejercicio 5. Análisis diferencial de expresión génica con DESeq2	34
4.1.1. Enunciado	34
4.1.2. Resolución	34

1. Introducción

La presente memoria recoge el desarrollo de la **PEC3** de la asignatura *Software para el Análisis de Datos*, perteneciente al **Máster en Bioinformática y Bioestadística** (UOC-UB). El objetivo principal de esta práctica es consolidar los conocimientos adquiridos en los **Laboratorios 5 y 6**, centrados en técnicas de **Machine Learning**, análisis estadístico y desarrollo de aplicaciones interactivas en **R**.

En el **Laboratorio 5** se trabajan conceptos relacionados con modelos de regresión, análisis de la varianza y técnicas de análisis de datos aplicadas mediante ejemplos prácticos en R. Por su parte, el **Laboratorio 6** introduce el uso de **Shiny** para la creación de aplicaciones web interactivas y el ecosistema **Bioconductor** para el análisis bioinformático y genómico.

A lo largo de esta memoria se aplican estos conceptos en ejercicios relacionados con clasificación, validación de modelos, análisis estadístico y análisis diferencial de expresión génica, reforzando competencias en **programación en R**, **modelización estadística** y **análisis de datos**.

El código utilizado en esta memoria se encuentra disponible en el repositorio de GitHub asociado al trabajo: [biostatistics-ml-r](#).

2. Sección 1: Modelos de clasificación y validación

2.1. Ejercicio 1. Clasificación de Tumores Mamarios con Breast Cancer Wisconsin

2.1.1. Enunciado

El conjunto de datos Breast Cancer Wisconsin, incluido en el paquete `mlbench` de R, es un dataset clásico de clasificación supervisada utilizado para predecir si un tumor mamario es benigno o maligno a partir de diversas características observadas en muestras celulares.

Estas características describen aspectos morfológicos de las células extraídas mediante una biopsia por aspiración con aguja fina del tejido mamario.

Cada fila del dataset representa una muestra clínica de una paciente, y las columnas recogen variables que miden propiedades de las células, como:

- grosor del grupo celular
- uniformidad del tamaño y forma
- adhesión marginal
- tamaño de células epiteliales
- tamaño de núcleos
- cromatina
- nucléolos
- número de mitosis

La variable respuesta es `Class`, que indica el diagnóstico final: benigno o maligno. Se pide:

- a) Cargar el dataset y realizar una exploración inicial (resumen de variables, valores perdidos, tratamiento aplicado, etc.).
- b) Dividir el dataset en dos subconjuntos:
 - Entrenamiento (70 %)
 - Test (30 %)

Explicar brevemente el procedimiento utilizado.

- c) Generar un modelo de clasificación:
 - Ajustar un modelo para predecir `Class`
 - Indicar el modelo utilizado y justificar la elección
- d) Validar el modelo con el conjunto de test:
 - Matriz de confusión
 - Métricas: accuracy, sensibilidad, especificidad, F1-score
 - Interpretación crítica

e) Comparar con una estrategia baseline:

Asignar siempre la clase mayoritaria (**benign**).

Código orientativo:

```
# install.packages("mlbench")

library(mlbench)

data("BreastCancer")
```

La accuracy baseline se calcula mediante la siguiente expresión:

$$\text{Accuracy baseline} = \frac{\text{nombre de casos de la clase mayoritaria}}{\text{nombre total de casos}}$$

2.1.2. Resolución

En este primer ejercicio de la PEC 3 se aborda la clasificación de tumores mamarios utilizando el dataset Breast Cancer Wisconsin incluido en el paquete **mlbench** de R. Se trata de un dataset clásico de clasificación supervisada empleado habitualmente en problemas de diagnóstico médico, cuyo objetivo es predecir si un tumor mamario es benigno o maligno a partir de distintas características observadas en muestras celulares obtenidas mediante biopsia por aspiración con aguja fina.

A continuación, se presenta la resolución paso a paso de cada apartado solicitado:

a)

En primer lugar, se cargan las librerías necesarias para la manipulación, exploración y visualización de los datos, así como el propio conjunto de datos objeto de estudio.

En este caso, el paquete **mlbench** proporciona datasets clásicos utilizados en problemas de *Machine Learning*. Por su parte, **tidyverse** facilita tareas de manipulación, transformación y visualización de datos, mientras que **skimr** permite generar resúmenes descriptivos más completos e interpretables.

El código para cargar las librerías y el dataset es el siguiente:

```
# Cargar librerías necesarias
library(mlbench)
library(tidyverse)
library(skimr)

# Cargar el dataset Breast Cancer Wisconsin
data("BreastCancer")
```

Una vez cargado el conjunto de datos, se realiza una primera inspección exploratoria con el objetivo de conocer su estructura, dimensiones y tipos de variables disponibles. Para ello, se emplean las funciones **dim()**, **str()** y **head()**, que permiten obtener información sobre el número de filas y columnas, la estructura de las variables y una vista previa de las primeras observaciones, respectivamente:

```
# Exploración inicial del dataset
dim(BreastCancer)
```

```
## [1] 699 11
```

```
str(BreastCancer)
```

```
## 'data.frame':    699 obs. of  11 variables:
## $ Id           : chr  "1000025" "1002945" "1015425" "1016277" ...
## $ Cl.thickness  : Ord.factor w/ 10 levels "1"<"2"<"3"<"4"<...: 5 5 3 6 4 8 1 2 2 4 ...
## $ Cell.size     : Ord.factor w/ 10 levels "1"<"2"<"3"<"4"<...: 1 4 1 8 1 10 1 1 1 2 ...
## $ Cell.shape    : Ord.factor w/ 10 levels "1"<"2"<"3"<"4"<...: 1 4 1 8 1 10 1 2 1 1 ...
## $ Marg.adhesion : Ord.factor w/ 10 levels "1"<"2"<"3"<"4"<...: 1 5 1 1 3 8 1 1 1 1 ...
## $ Epith.c.size  : Ord.factor w/ 10 levels "1"<"2"<"3"<"4"<...: 2 7 2 3 2 7 2 2 2 2 ...
## $ Bare.nuclei   : Factor w/ 10 levels "1","2","3","4",...: 1 10 2 4 1 10 10 1 1 1 ...
## $ Bl.cromatin    : Factor w/ 10 levels "1","2","3","4",...: 3 3 3 3 3 9 3 3 1 2 ...
## $ Normal.nucleoli: Factor w/ 10 levels "1","2","3","4",...: 1 2 1 7 1 7 1 1 1 1 ...
## $ Mitoses       : Factor w/ 9 levels "1","2","3","4",...: 1 1 1 1 1 1 1 1 5 1 ...
## $ Class         : Factor w/ 2 levels "benign","malignant": 1 1 1 1 1 2 1 1 1 1 ...
```

```
head(BreastCancer)
```

```
##      Id Cl.thickness Cell.size Cell.shape Marg.adhesion Epith.c.size
## 1 1000025          5         1         1           1           2
## 2 1002945          5         4         4           5           7
## 3 1015425          3         1         1           1           2
## 4 1016277          6         8         8           1           3
## 5 1017023          4         1         1           3           2
## 6 1017122          8        10        10           8           7
##  Bare.nuclei Bl.cromatin Normal.nucleoli Mitoses      Class
## 1           1           3           1           1    benign
## 2          10           3           2           1    benign
## 3           2           3           1           1    benign
## 4           4           3           7           1    benign
## 5           1           3           1           1    benign
## 6          10           9           7           1 malignant
```

Como se puede observar en el output, el conjunto de datos contiene inicialmente **699 observaciones** y **11 variables**. Cada fila representa una muestra clínica correspondiente a una paciente, mientras que las columnas describen distintas características morfológicas de las células analizadas, tales como el grosor del grupo celular, la uniformidad del tamaño y forma celular, la adhesión marginal o el tamaño de los núcleos. Por su parte, la variable respuesta del problema es **Class**, encargada de indicar el diagnóstico final del tumor, clasificándolo como benigno (**benign**) o maligno (**malignant**).

A partir de esta primera inspección puede observarse que la mayoría de variables se encuentran almacenadas como factores ordinales. Aunque representan escalas discretas comprendidas entre 1 y

10, el dataset original las codifica como variables categóricas ordenadas. Asimismo, se identifica la existencia de una variable denominada `Id`, utilizada únicamente como identificador de cada muestra y que, por tanto, no aporta información útil desde el punto de vista predictivo.

Con el objetivo de obtener una visión descriptiva más completa del conjunto de datos, se calcula a continuación un resumen estadístico de las variables.

```
# Resumen estadístico de las variables
skim(BreastCancer)
```

Cuadro 1: Data summary

Name	BreastCancer
Number of rows	699
Number of columns	11
Column type frequency:	
character	1
factor	10
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
Id	0	1	5	8	0	645	0

Variable type: factor

skim_variable	n_missing	complete_rate	ordered	n_unique	top_counts
Cl.thickness	0	1.00	TRUE	10	1: 145, 5: 130, 3: 108, 4: 80
Cell.size	0	1.00	TRUE	10	1: 384, 10: 67, 3: 52, 2: 45
Cell.shape	0	1.00	TRUE	10	1: 353, 2: 59, 10: 58, 3: 56
Marg.adhesion	0	1.00	TRUE	10	1: 407, 2: 58, 3: 58, 10: 55
Epith.c.size	0	1.00	TRUE	10	2: 386, 3: 72, 4: 48, 1: 47
Bare.nuclei	16	0.98	FALSE	10	1: 402, 10: 132, 2: 30, 5: 30
Bl.cromatin	0	1.00	FALSE	10	2: 166, 3: 165, 1: 152, 7: 73
Normal.nucleoli	0	1.00	FALSE	10	1: 443, 10: 61, 3: 44, 2: 36
Mitoses	0	1.00	FALSE	9	1: 579, 2: 35, 3: 33, 10: 14
Class	0	1.00	FALSE	2	ben: 458, mal: 241

El análisis descriptivo permite observar la distribución de las distintas variables y detectar posibles incidencias relacionadas con la calidad de los datos. En este caso, tanto `summary()` como `skim()` muestran que la variable `Bare.nuclei` presenta valores ausentes. Concretamente, `summary()` indica

la presencia de 16 valores NA, mientras que `skim()` refleja igualmente 16 observaciones perdidas en dicha variable.

Para confirmarlo de forma explícita, se comprueba el número total de valores ausentes en cada columna del conjunto de datos.

```
# Comprobar valores ausentes por columna
colSums(is.na(BreastCancer))
```

```
##           Id      Cl.thickness      Cell.size      Cell.shape      Marg.adhesion
##           0           0           0           0           0
##  Epith.c.size      Bare.nuclei      Bl.cromatin      Normal.nucleoli      Mitoses
##           0           16           0           0           0
##           Class
##           0
```

Se observa que únicamente la variable `Bare.nuclei` presenta valores faltantes, concretamente en 16 observaciones. Debido a que esta cantidad representa un porcentaje reducido respecto al tamaño total de la muestra, se decide eliminar dichas filas mediante la función `na.omit()`, ya que apenas reduce el tamaño muestral disponible y evita introducir posibles sesgos derivados de técnicas de imputación.

```
BreastCancer <- na.omit(BreastCancer)

dim(BreastCancer)
```

```
## [1] 683  11
```

Tras la eliminación de observaciones incompletas, el conjunto de datos pasa a contener **683 observaciones válidas**.

A continuación, se realiza una fase básica de preprocesado orientada a preparar los datos para futuros modelos de clasificación. En primer lugar, se elimina la variable `Id`, ya que su función es exclusivamente identificativa y no contiene información relevante para el análisis predictivo.

```
# Eliminar la variable Id
BreastCancer <- BreastCancer %>%
  select(-Id)
```

Posteriormente, las variables predictoras se convierten a formato numérico, ya que, aunque originalmente aparecen codificadas como factores ordinales, representan escalas discretas entre 1 y 10, por lo que resulta más conveniente tratarlas como variables numéricas de cara a los análisis posteriores.

```
# Convertir variables predictoras a formato numérico
BreastCancer <- BreastCancer %>%
  mutate(across(-Class, ~as.numeric(as.character(.))))
```


Finalmente, se verifica nuevamente la estructura del conjunto de datos tras el proceso de limpieza y transformación aplicado.

```
# Verificar la estructura del dataset tras preprocesado
str(BreastCancer)
```

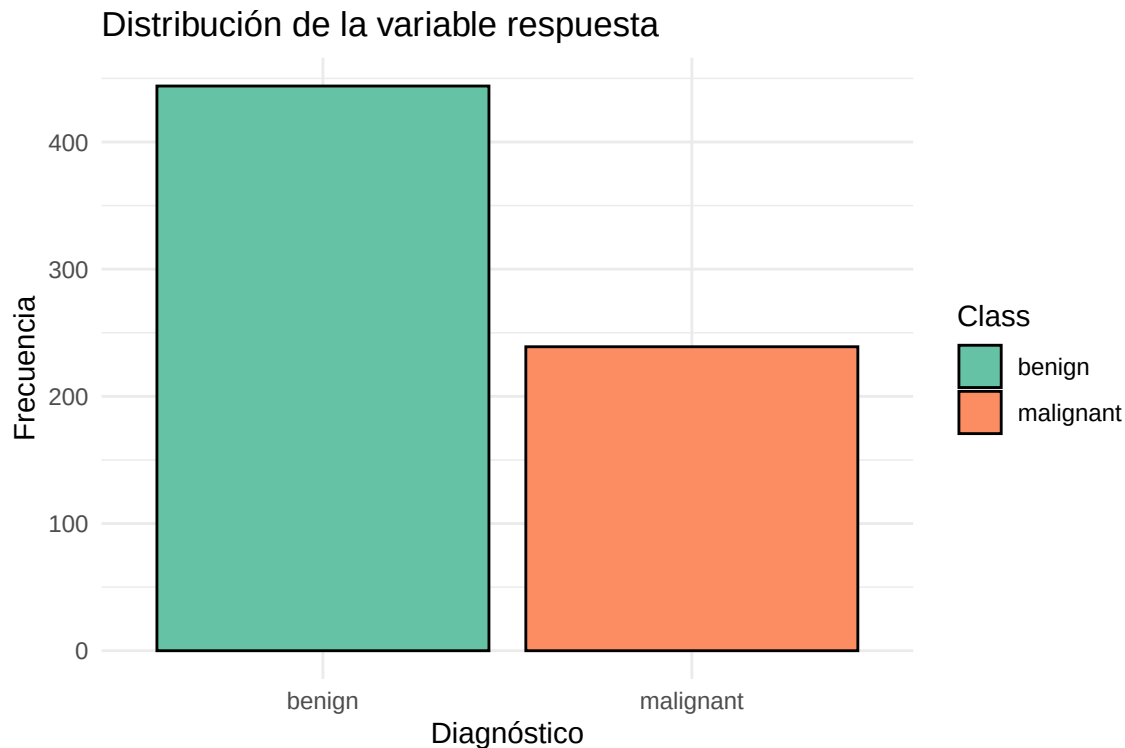
```
## 'data.frame':    683 obs. of  10 variables:
## $ Cl.thickness    : num  5 5 3 6 4 8 1 2 2 4 ...
## $ Cell.size       : num  1 4 1 8 1 10 1 1 1 2 ...
## $ Cell.shape      : num  1 4 1 8 1 10 1 2 1 1 ...
## $ Marg.adhesion   : num  1 5 1 1 3 8 1 1 1 1 ...
## $ Epith.c.size    : num  2 7 2 3 2 7 2 2 2 2 ...
## $ Bare.nuclei     : num  1 10 2 4 1 10 10 1 1 1 ...
## $ Bl.cromatin      : num  3 3 3 3 3 9 3 3 1 2 ...
## $ Normal.nucleoli : num  1 2 1 7 1 7 1 1 1 1 ...
## $ Mitoses         : num  1 1 1 1 1 1 1 1 5 1 ...
## $ Class           : Factor w/ 2 levels "benign","malignant": 1 1 1 1 1 2 1 1 1 1 ...
## - attr(*, "na.action")= 'omit' Named int [1:16] 24 41 140 146 159 165 236 250 276 293 ...
## ..- attr(*, "names")= chr [1:16] "24" "41" "140" "146" ...
```

```
summary(BreastCancer)
```

```
##   Cl.thickness      Cell.size      Cell.shape      Marg.adhesion
## Min.   : 1.000      Min.   : 1.000      Min.   : 1.000      Min.   : 1.00
## 1st Qu.: 2.000      1st Qu.: 1.000      1st Qu.: 1.000      1st Qu.: 1.00
## Median : 4.000      Median : 1.000      Median : 1.000      Median : 1.00
## Mean   : 4.442      Mean   : 3.151      Mean   : 3.215      Mean   : 2.83
## 3rd Qu.: 6.000      3rd Qu.: 5.000      3rd Qu.: 5.000      3rd Qu.: 4.00
## Max.   :10.000      Max.   :10.000      Max.   :10.000      Max.   :10.00
##   Epith.c.size      Bare.nuclei      Bl.cromatin      Normal.nucleoli
## Min.   : 1.000      Min.   : 1.000      Min.   : 1.000      Min.   : 1.00
## 1st Qu.: 2.000      1st Qu.: 1.000      1st Qu.: 2.000      1st Qu.: 1.00
## Median : 2.000      Median : 1.000      Median : 3.000      Median : 1.00
## Mean   : 3.234      Mean   : 3.545      Mean   : 3.445      Mean   : 2.87
## 3rd Qu.: 4.000      3rd Qu.: 6.000      3rd Qu.: 5.000      3rd Qu.: 4.00
## Max.   :10.000      Max.   :10.000      Max.   :10.000      Max.   :10.00
##      Mitoses          Class
## Min.   : 1.000      benign   :444
## 1st Qu.: 1.000      malignant:239
## Median : 1.000
## Mean   : 1.603
## 3rd Qu.: 1.000
## Max.   :10.000
```

Como parte de la exploración inicial, también se analiza la distribución de la variable respuesta para comprobar si existe un posible desbalance entre clases. Para ello, se genera un gráfico de barras que muestra la frecuencia de cada categoría dentro de la variable `Class`.

```
# Gráfico de barras para la variable respuesta
ggplot(BreastCancer, aes(x = Class, fill = Class)) +
  geom_bar(color = "black") +
  theme_minimal() +
  labs(
    title = "Distribución de la variable respuesta",
    x = "Diagnóstico",
    y = "Frecuencia"
  ) +
  scale_fill_brewer(palette = "Set2")
```



Como se puede observar en el gráfico obtenido, se muestra que ambas categorías presentan una representación suficiente dentro del conjunto de datos, aunque la clase correspondiente a tumores benignos aparece con una frecuencia ligeramente superior. En cualquier caso, no se aprecia un desbalance extremo que pueda comprometer significativamente el rendimiento de futuros modelos predictivos.

b)

Una vez finalizada la fase de exploración y preprocesado, se procede a dividir el conjunto de datos en dos subconjuntos independientes: un conjunto de entrenamiento (*training set*) y un conjunto de prueba (*test set*).

Esta división es fundamental en problemas de *Machine Learning*, ya que permite entrenar el modelo utilizando únicamente una parte de los datos y evaluar posteriormente su capacidad de generalización sobre observaciones no vistas previamente.

Para realizar esta partición se utiliza la función `createDataPartition()` del paquete `caret`. Esta

función permite generar particiones estratificadas, conservando aproximadamente la misma proporción de clases de la variable respuesta (**Class**) tanto en el conjunto de entrenamiento como en el de test. En este caso, se establece una división del 70 % de las observaciones para entrenamiento y del 30 % para evaluación o test. Además, se fija una semilla mediante `set.seed()` con el objetivo de garantizar la reproducibilidad de los resultados:

```
# División del dataset en entrenamiento y test
library(caret)

set.seed(123)

train_index <- createDataPartition(
  BreastCancer$Class,
  p = 0.7,
  list = FALSE
)

train_data <- BreastCancer[train_index, ]
test_data  <- BreastCancer[-train_index, ]
```

A continuación, se comprueba el número de observaciones presentes en cada subconjunto generado:

```
# Comprobar dimensiones de los subconjuntos
dim(train_data)

## [1] 479 10

dim(test_data)
```

```
## [1] 204 10
```

Como se puede observar, el conjunto de entrenamiento contiene 479 observaciones, mientras que el conjunto de test cuenta con 204 observaciones. Esta distribución corresponde aproximadamente al 70 % y 30 % del total de datos disponibles, respectivamente.

Finalmente, se verifica que la distribución de la variable respuesta se mantiene de forma similar en ambos subconjuntos:

```
# Verificar distribución de la variable respuesta en ambos subconjuntos
prop.table(table(train_data$Class))

##
##      benign malignant
## 0.6492693 0.3507307
```

```
prop.table(table(test_data$Class))
```

```
##  
##      benign malignant  
## 0.6519608 0.3480392
```

Los resultados obtenidos muestran que la proporción de tumores benignos y malignos se conserva de manera equilibrada tanto en el conjunto de entrenamiento como en el conjunto de test. Esto es importante en problemas de clasificación, ya que evita introducir sesgos derivados de distribuciones descompensadas entre ambos subconjuntos.

c)

Una vez realizada la partición del conjunto de datos, se procede a construir un modelo de clasificación capaz de predecir la variable respuesta **Class**. Es decir, un modelo que sea capaz de determinar si un tumor es benigno (**benign**) o maligno (**malignant**) a partir de las características morfológicas observadas en las muestras celulares.

En este caso, se ha optado por utilizar un modelo de tipo *Random Forest*. Este algoritmo pertenece a la familia de métodos basados en árboles de decisión y constituye una de las técnicas más utilizadas en problemas de clasificación supervisada debido a su elevada capacidad predictiva y robustez frente al sobreajuste.

La elección de este modelo se debe a que encaja bien para abordar este problema, ya que el dataset contiene múltiples variables morfológicas potencialmente relacionadas entre sí y con posibles relaciones no lineales. En este contexto, *Random Forest* permite combinar numerosos árboles de decisión independientes, mejorando la capacidad de generalización del modelo y reduciendo la sensibilidad frente a variaciones concretas de los datos.

Para ajustar el modelo se utiliza el paquete **randomForest**. A continuación, se muestra el código empleado para construir el modelo de clasificación:

```
# Ajuste del modelo de Random Forest
library(randomForest)

set.seed(123)

modelo_rf <- randomForest(
  Class ~ .,
  data = train_data,
  ntree = 500,
  importance = TRUE
)
```

En este caso, el parámetro **ntree = 500** indica que el modelo construirá 500 árboles de decisión independientes, combinando posteriormente sus predicciones mediante votación mayoritaria. En este caso, se ha optado por un número de árboles relativamente elevado para garantizar una buena capacidad de generalización, aunque en la práctica podría ser necesario ajustar este parámetro mediante técnicas de validación cruzada para optimizar el rendimiento del modelo.

Por su parte, el argumento **importance = TRUE** permite calcular la importancia relativa de cada variable predictora dentro del modelo, lo cual resulta útil para identificar qué características morfológicas contribuyen en mayor medida a la clasificación de los tumores.

Una vez ajustado el modelo, se examina un resumen general del mismo:

```
# Resumen del modelo de Random Forest
modelo_rf

##
## Call:
## randomForest(formula = Class ~ ., data = train_data, ntree = 500,      importance = TRUE)
```

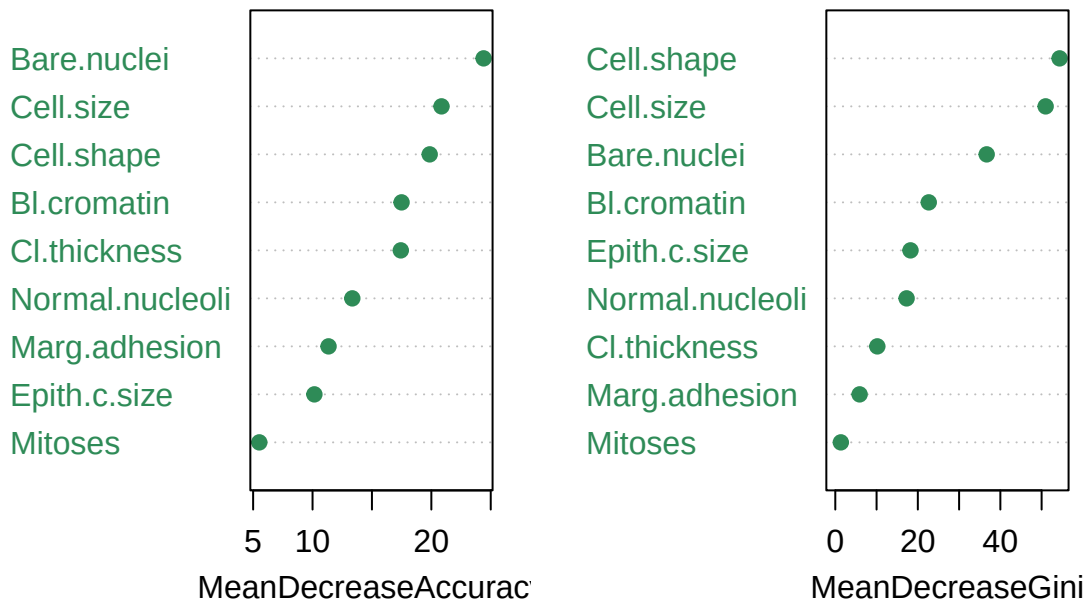
```
##                Type of random forest: classification
##                Number of trees: 500
## No. of variables tried at each split: 3
##
##                OOB estimate of  error rate: 2.92%
## Confusion matrix:
##                benign malignant class.error
## benign         303           8 0.02572347
## malignant       6          162 0.03571429
```

A la vista de los resultados obtenidos, se observa que el modelo de *Random Forest* obtiene una tasa de error OOB (out-of-bag) del 2.92%. Esta métrica se calcula internamente durante el entrenamiento mediante las observaciones que no han sido utilizadas para construir cada árbol, por lo que proporciona una estimación interna del error de generalización. No obstante, la evaluación definitiva del rendimiento debe realizarse sobre el conjunto de test reservado previamente.

Asimismo, dada la naturaleza del algoritmo de *Random Forest*, es posible analizar la importancia relativa de las variables predictoras dentro del modelo. En este caso, la importancia de variables se calcula mediante dos métricas principales: el aumento de la precisión (Mean Decrease Accuracy) y la disminución de la impureza (Mean Decrease Gini).

```
# Gráfico de importancia de variables
varImpPlot(
  modelo_rf,
  main = "Importancia de las variables",
  pch = 19,
  col = "#2E8B57")
```

Importancia de las variables



El gráfico de importancia de variables permite observar que las características relacionadas con la morfología celular son las que tienen mayor peso en la clasificación de los tumores. En concreto, variables como `Bare.nuclei`, `Cell.size` y `Cell.shape` destacan por su elevada contribución al modelo.

La métrica `MeanDecreaseAccuracy` muestra cómo afecta cada variable a la precisión del modelo, mientras que `MeanDecreaseGini` indica su capacidad para mejorar la separación entre clases dentro de los árboles de decisión. En ambos casos, los resultados sugieren que las alteraciones en el tamaño, la forma y la estructura nuclear son factores especialmente relevantes para diferenciar tumores benignos y malignos.

d)

Una vez entrenado el modelo de *Random Forest*, se procede a evaluar su capacidad de generalización utilizando el conjunto de test. Para ello, en primer lugar se generan las predicciones sobre el conjunto de test mediante la función `predict()`:

```
# Generar predicciones sobre el conjunto de test
predicciones <- predict(modelo_rf, newdata = test_data)
```

A continuación, se construye la matriz de confusión, que permite comparar las clases reales con las predichas por el modelo:

```
# Matriz de confusión considerando malignant como clase positiva
cm <- confusionMatrix(
  data = predicciones,
  reference = test_data$Class,
  positive = "malignant"
)

cm
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction  benign malignant
##   benign      130         2
##   malignant     3         69
##
##              Accuracy : 0.9755
##              95% CI : (0.9437, 0.992)
##   No Information Rate : 0.652
##   P-Value [Acc > NIR] : <2e-16
##
##              Kappa : 0.9462
##
##  Mcnemar's Test P-Value : 1
##
##              Sensitivity : 0.9718
##              Specificity : 0.9774
##              Pos Pred Value : 0.9583
##              Neg Pred Value : 0.9848
##              Prevalence : 0.3480
##              Detection Rate : 0.3382
##              Detection Prevalence : 0.3529
##              Balanced Accuracy : 0.9746
##
##              'Positive' Class : malignant
##
```

Asimismo, se calculan las métricas principales de evaluación del modelo, incluyendo la accuracy, sensibilidad, especificidad, precisión y F1-score:

```
# Extraer métricas principales
accuracy <- cm$overall["Accuracy"]
sensibilidad <- cm$byClass["Sensitivity"]
especificidad <- cm$byClass["Specificity"]
precision <- cm$byClass["Precision"]
f1_score <- cm$byClass["F1"]
```



```
accuracy
```

```
## Accuracy  
## 0.9754902
```

```
sensibilidad
```

```
## Sensitivity  
## 0.971831
```

```
especificidad
```

```
## Specificity  
## 0.9774436
```

```
precision
```

```
## Precision  
## 0.9583333
```

```
f1_score
```

```
## F1  
## 0.965035
```

En este caso se ha considerado la clase **malignant** como clase positiva, ya que en un contexto biomédico el objetivo debería centrarse en evaluar la capacidad del modelo para detectar correctamente los tumores malignos. En este caso, la matriz de confusión obtenida muestra que el modelo clasifica correctamente la mayoría de observaciones del conjunto de test, con un número muy reducido de falsos positivos y falsos negativos. Por su parte, la accuracy obtenida es aproximadamente del 97.5 %, lo que indica un rendimiento global muy elevado. Además, la sensibilidad para la clase **malignant** es alta, lo que significa que el modelo identifica correctamente la mayoría de tumores malignos. La especificidad también es elevada, indicando que el modelo clasifica correctamente la mayoría de tumores benignos. Finalmente, el valor de F1-score, cercano a 0.97, confirma que el modelo mantiene un buen equilibrio entre precisión y sensibilidad para la detección de tumores malignos. Esto es especialmente relevante en problemas médicos, donde los falsos negativos pueden tener consecuencias importantes.

e)

Con el objetivo de valorar adecuadamente el rendimiento del modelo construido, se compara con una estrategia **baseline** sencilla. En este caso, la estrategia propuesta consiste en clasificar todas las

observaciones como pertenecientes a la clase mayoritaria observada en el conjunto de entrenamiento. Posteriormente, dichas predicciones baseline se evalúan sobre el conjunto de test, permitiendo comparar ambos enfoques sobre el mismo conjunto de evaluación.

En primer lugar, se identifica cuál es la clase mayoritaria en el conjunto de entrenamiento:

```
# Clase mayoritaria en el conjunto de entrenamiento
clase_mayoritaria <- names(which.max(table(train_data$Class)))

clase_mayoritaria
```

```
## [1] "benign"
```

Puede observarse que la clase mayoritaria corresponde a `benign`.

A continuación, se generan predicciones baseline asignando dicha clase a todas las observaciones del conjunto de test:

```
# Predicciones baseline sobre el conjunto de test
baseline_pred <- factor(
  rep(clase_mayoritaria, nrow(test_data)),
  levels = levels(test_data$Class)
)
```

Posteriormente, se evalúa el rendimiento de esta estrategia mediante una matriz de confusión:

```
# Evaluación del modelo baseline
baseline_cm <- confusionMatrix(
  data = baseline_pred,
  reference = test_data$Class,
  positive = "malignant"
)

baseline_cm
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction  benign malignant
##   benign      133         71
##   malignant     0          0
##
##              Accuracy : 0.652
##              95% CI : (0.5823, 0.7171)
##   No Information Rate : 0.652
##   P-Value [Acc > NIR] : 0.5322
##
```

```

##                Kappa : 0
##
##  McNemar's Test P-Value : <2e-16
##
##                Sensitivity : 0.000
##                Specificity : 1.000
##                Pos Pred Value :  NaN
##                Neg Pred Value : 0.652
##                Prevalence : 0.348
##                Detection Rate : 0.000
##  Detection Prevalence : 0.000
##  Balanced Accuracy : 0.500
##
##  'Positive' Class : malignant
##

```

La estrategia baseline, basada en clasificar todas las observaciones como **benign**, obtiene una accuracy aproximada del 65 %, valor similar a la proporción de casos benignos presentes en el conjunto de datos. Sin embargo, esta estrategia no tiene capacidad real de discriminación, ya que clasifica todos los casos como benignos y, por tanto, no detecta ningún tumor maligno. Esto implica una sensibilidad nula para la clase malignant, lo que la convierte en una estrategia claramente insuficiente desde el punto de vista clínico.

En comparación, el modelo de *Random Forest* alcanza una accuracy cercana al 97.5 % y presenta además valores elevados de sensibilidad, especificidad y F1-score para la detección de tumores malignos. Esto demuestra que el modelo aprende patrones relevantes de las variables predictoras y ofrece un rendimiento claramente superior al de la estrategia baseline. Esta comparación demuestra que, en problemas biomédicos, no es suficiente evaluar únicamente la accuracy, sino que resulta fundamental considerar métricas adicionales relacionadas con la detección correcta de los casos clínicamente relevantes.

2.2. Ejercicio 2. Humedad del suelo y cobertura vegetal

2.2.1. Enunciado

Se midió la humedad del suelo bajo diferentes coberturas vegetales:

- H1: hierba baja
- H2: arbustos
- H3: arbolado disperso
- H4: bosque denso

Valores:

- H1: 28, 33, 30, 27, 31, 28
- H2: 35, 37, 36, 38, 34, 33
- H3: 40, 39, 41, 42, 40, 43

- H4: 48, 46, 47, 45, 44, 48

Se pide:

- Reformule los datos en un `data.frame` en formato largo (*tidy data*), de modo que exista una columna numérica con la variable humedad y una columna de tipo factor con la variable cubierta. Es necesario mostrar la estructura final del conjunto de datos.
- Construye con `ggplot2` un gráfico que permita comparar la distribución de la humedad del suelo entre los diferentes tipos de cubierta. A continuación, interpreta brevemente el gráfico y valora si, visualmente, parece que existen diferencias entre grupos.
- Ajustar un modelo ANOVA para estudiar si la humedad del suelo varía significativamente según el tipo de cubierta vegetal. Habrá que especificar el modelo ajustado, los principales resultados obtenidos e interpretación de éstos.
- Si el resultado del ANOVA es significativo, aplica una prueba post-hoc de Tukey (*Tukey HSD*) para identificar entre qué grupos existen diferencias significativas.
- Validar los supuestos del modelo ANOVA mediante gráficos de residuos y comentar, de forma razonada, si los supuestos de normalidad y homogeneidad de varianzas parecen aceptables.

2.2.2. Resolución

En este segundo ejercicio se analiza si la humedad del suelo varía significativamente en función del tipo de cobertura vegetal presente. Para ello, se emplea un análisis de la varianza (*ANOVA*), técnica estadística utilizada para comparar medias entre varios grupos independientes.

a)

En primer lugar, se construye un conjunto de datos en formato largo (*tidy data*), donde cada fila representa una observación individual de humedad del suelo y cada observación queda asociada a su correspondiente tipo de cubierta vegetal.

```
# Cargar librerías necesarias
library(tidyverse)

# Construir el data.frame en formato largo
humedad <- c(
  28, 33, 30, 27, 31, 28,
  35, 37, 36, 38, 34, 33,
  40, 39, 41, 42, 40, 43,
  48, 46, 47, 45, 44, 48
)

# Crear la variable de tipo factor para la cubierta vegetal
cubierta <- factor(rep(
  c("H1", "H2", "H3", "H4"),
  each = 6
))
```

```

))

# Combinar en un data.frame
datos_humedad <- data.frame(
  humedad,
  cubierta
)

```

A continuación, se muestra la estructura final del conjunto de datos:

```

# Mostrar la estructura del data.frame
str(datos_humedad)

## 'data.frame':    24 obs. of  2 variables:
## $ humedad : num  28 33 30 27 31 28 35 37 36 38 ...
## $ cubierta: Factor w/ 4 levels "H1","H2","H3",...: 1 1 1 1 1 1 2 2 2 2 ...

```

```
head(datos_humedad)
```

```

##   humedad cubierta
## 1      28      H1
## 2      33      H1
## 3      30      H1
## 4      27      H1
## 5      31      H1
## 6      28      H1

```

El conjunto de datos resultante contiene una variable numérica denominada `humedad`, correspondiente a los niveles de humedad medidos en el suelo, y una variable de tipo factor llamada `cubierta`, que identifica el tipo de cobertura vegetal asociado a cada observación.

b)

Una vez organizados los datos, se construye un gráfico utilizando `ggplot2` con el objetivo de comparar visualmente la distribución de la humedad entre los distintos tipos de cubierta vegetal.

```

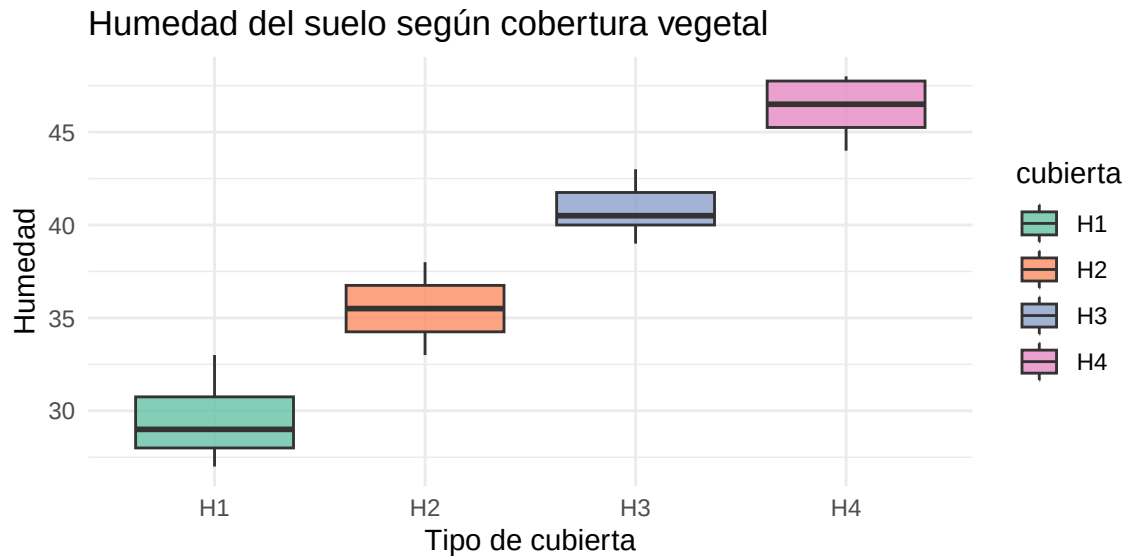
# Gráfico de boxplot para comparar la humedad entre grupos
ggplot(datos_humedad,
  aes(x = cubierta,
      y = humedad,
      fill = cubierta)) +
  geom_boxplot(alpha = 0.8) +
  theme_minimal() +
  labs(
    title = "Humedad del suelo según cobertura vegetal",
    x = "Tipo de cubierta",

```

```

y = "Humedad"
) +
scale_fill_brewer(palette = "Set2")

```



El gráfico muestra una tendencia claramente creciente en los niveles de humedad del suelo a medida que aumenta la densidad de la cobertura vegetal. En concreto, el grupo H1 presenta los valores más bajos de humedad, mientras que H4 registra los valores más elevados. Además, visualmente se aprecia una separación bastante clara entre grupos, lo que sugiere la posible existencia de diferencias significativas en las medias de humedad según el tipo de cubierta vegetal.

c)

Con el objetivo de estudiar formalmente si existen diferencias significativas entre grupos, se ajusta un modelo ANOVA de un factor, considerando la humedad como variable respuesta y el tipo de cubierta vegetal como variable explicativa.

El modelo ajustado es el siguiente:

$$\text{humedad}_{ij} = \mu + \alpha_i + \varepsilon_{ij}$$

donde:

- μ representa la media global.
- α_i corresponde al efecto asociado a cada tipo de cubierta vegetal.
- ε_{ij} representa el término de error aleatorio.

El ajuste del modelo se realiza mediante la función `aov()`:

```

# Ajuste del modelo ANOVA
modelo_anova <- aov(
  humedad ~ cubierta,

```

```
data = datos_humedad
)

summary(modelo_anova)
```

```
##           Df Sum Sq Mean Sq F value    Pr(>F)
## cubierta     3  935.8   311.93    92.88 6.51e-12 ***
## Residuals    20   67.2     3.36
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Los resultados obtenidos muestran un valor de *p-value* muy inferior a 0.05, lo que indica que existen diferencias estadísticamente significativas entre las medias de humedad del suelo según el tipo de cobertura vegetal considerado.

Por tanto, puede rechazarse la hipótesis nula de igualdad de medias y concluir que el tipo de cubierta vegetal influye significativamente sobre la humedad del suelo.

d)

Dado que el resultado del ANOVA es significativo, se procede a realizar una prueba post-hoc de Tukey (*Tukey HSD*) para identificar específicamente entre qué grupos existen diferencias significativas.

```
# Prueba post-hoc de Tukey
TukeyHSD(modelo_anova)
```

```
##      Tukey multiple comparisons of means
##      95% family-wise confidence level
##
## Fit: aov(formula = humedad ~ cubierta, data = datos_humedad)
##
## $cubierta
##           diff           lwr          upr          p adj
## H2-H1  6.000000  3.038619  8.961381 0.0000825
## H3-H1 11.333333  8.371952 14.294714 0.0000000
## H4-H1 16.833333 13.871952 19.794714 0.0000000
## H3-H2  5.333333  2.371952  8.294714 0.0003380
## H4-H2 10.833333  7.871952 13.794714 0.0000000
## H4-H3  5.500000  2.538619  8.461381 0.0002368
```

Los resultados muestran diferencias significativas entre prácticamente todos los pares de grupos comparados. En general, la humedad aumenta progresivamente desde H1 hasta H4, observándose diferencias especialmente marcadas entre los grupos extremos (H1 y H4).

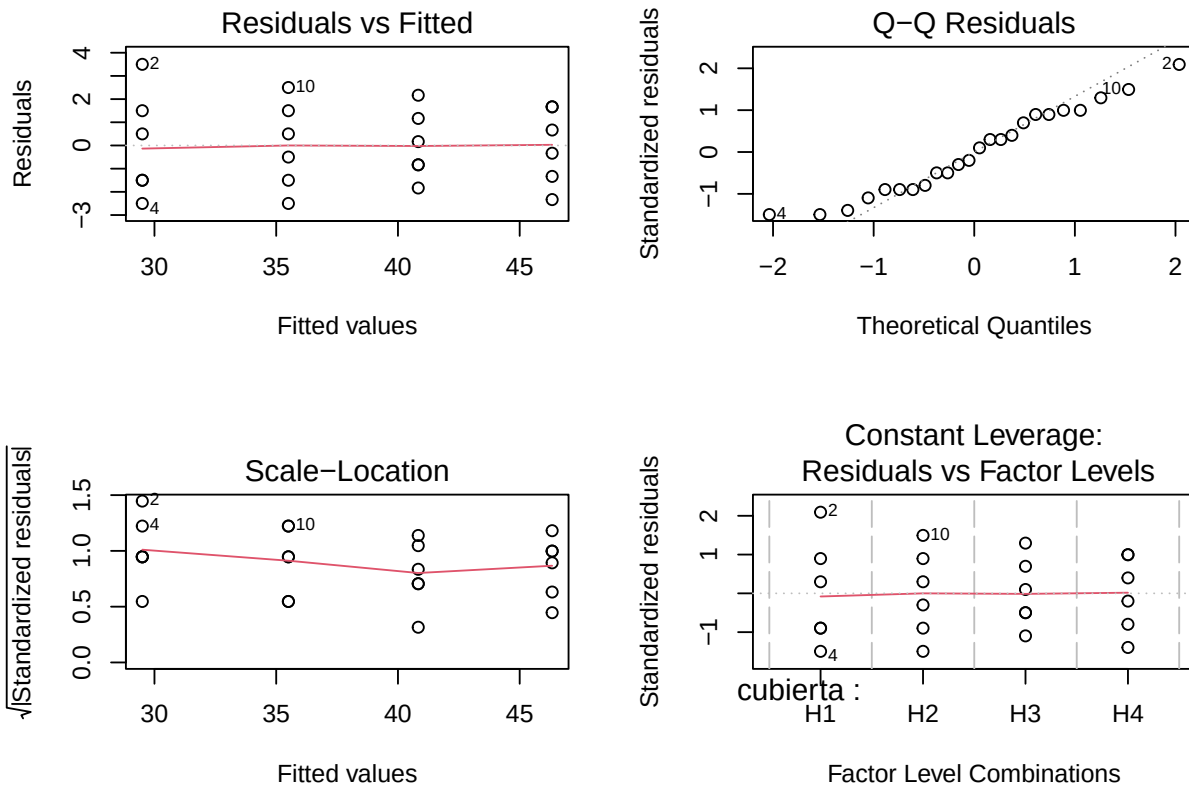
Estos resultados refuerzan la evidencia obtenida previamente en el ANOVA y sugieren que el incremento de densidad vegetal se asocia con mayores niveles de humedad en el suelo.

e)

Finalmente, se validan los supuestos básicos del modelo ANOVA mediante el análisis gráfico de residuos.

```
# Gráficos de diagnóstico para validar supuestos del modelo ANOVA
par(mfrow = c(2, 2))

plot(modelo_anova)
```



```
# Test de normalidad de los residuos
shapiro.test(residuals(modelo_anova))
```

```
##
## Shapiro-Wilk normality test
##
## data: residuals(modelo_anova)
## W = 0.95758, p-value = 0.3918
```



```
# Test de homogeneidad de varianzas
library(car)

leveneTest(humedad ~ cubierta, data = datos_humedad)

## Levene's Test for Homogeneity of Variance (center = median)
##           Df F value Pr(>F)
## group    3  0.5719   0.64
##           20
```

A partir de los gráficos de diagnóstico puede observarse que los residuos presentan una distribución aproximadamente normal y no se aprecian patrones claros de heterocedasticidad o dependencia. Asimismo, la dispersión de residuos parece relativamente homogénea entre grupos.

Además de la inspección gráfica, se ha aplicado el test de Shapiro-Wilk para evaluar la normalidad de los residuos del modelo. Si el *p-value* obtenido es superior a 0.05, no se rechaza la hipótesis nula de normalidad, por lo que los residuos pueden considerarse aproximadamente normales. Asimismo, se ha implementado el test de Levene para comprobar la homogeneidad de varianzas entre grupos. Un *p-value* superior a 0.05 indica que no existen evidencias suficientes para rechazar la igualdad de varianzas, por lo que el supuesto de homocedasticidad puede considerarse razonablemente aceptable.

En conjunto, tanto la inspección gráfica como los contrastes estadísticos sugieren que los supuestos principales del modelo ANOVA se cumplen de forma razonable, por lo que el modelo ajustado puede considerarse adecuado para este análisis.

3. Sección 2: Aplicaciones con Shiny y ética

3.1. Ejercicio 3. Aplicación interactiva con Shiny

3.1.1. Enunciado

En este ejercicio se propone el desarrollo de una aplicación interactiva con Shiny para explorar y filtrar el conjunto de datos `iris`, paquete incluido en R.

- a) Cargar los paquetes necesarios para desarrollar una aplicación Shiny que permita generar gráficos, mostrar resultados en formato tabla, y descargar datos filtrados. Construya una interfaz con un título principal adecuado y una disposición clara basada en:

- una zona lateral con los controles de entrada
- una zona principal con los resultados

- b) La interfaz debe incluir los siguientes elementos:

- un selector para elegir la variable que se representará en el eje X del gráfico, que contenga únicamente las variables numéricas del conjunto de datos `iris`.

- un segundo selector para elegir la variable que se representará en el eje Y.
- un tercer control para filtrar las observaciones según la especie, con las opciones: todas las observaciones, sólo setosa, sólo versicolor, sólo virginica.
- una casilla de selección que permita decidir si se desea mostrar o no una línea de regresión sobre el diagrama de dispersión.

c) Queremos que la aplicación muestre:

- un diagrama de dispersión con las variables seleccionadas en los ejes X e Y.
- puntos diferenciados según la especie.
- un título dinámico que indique qué variables se están comparando.
- un resumen estadístico del conjunto de datos filtrado.
- la posibilidad de descargar los datos filtrados en formato CSV.

d) Dentro del documento de respuesta, explicad brevemente cuál es la función de los siguientes elementos dentro de una aplicación Shiny: `ui` / `server` / `reactive`.

3.1.2. Resolución

En este ejercicio se desarrolla una aplicación interactiva avanzada con **Shiny** utilizando el conjunto de datos **iris**, uno de los datasets clásicos incluidos en R. El objetivo principal es construir una interfaz moderna e interactiva que permita explorar visualmente los datos, filtrar observaciones según la especie y generar resultados dinámicos en tiempo real.

La aplicación incorpora mejoras visuales mediante temas personalizados, paneles estilizados y navegación mediante pestañas para mejorar la experiencia del usuario.

a)

En primer lugar, se cargan las librerías necesarias para desarrollar la aplicación.

- **shiny** permite construir aplicaciones web interactivas en R.
- **ggplot2** se utiliza para generar gráficos.
- **DT** permite mostrar tablas interactivas.
- **dplyr** facilita el filtrado y manipulación de datos.
- **shinythemes** permite aplicar temas visuales modernos a la aplicación.

```
# Cargar librerías necesarias
library(shiny)
library(ggplot2)
library(DT)
library(dplyr)
library(shinythemes)
```

A continuación, se construye la interfaz gráfica de la aplicación mediante `fluidPage()`. En esta versión se incorpora:

- un tema visual (**flatly**),

- estilos CSS personalizados,
- paneles con sombras y bordes redondeados,
- una distribución más limpia mediante `fluidRow()` y `column()`,
- navegación por pestañas usando `tabsetPanel()`.

```
# Construcción de la interfaz gráfica (UI)
ui <- fluidPage(

  theme = shinytheme("flatly"),

  tags$head(
    tags$style(HTML("

      body {
        background-color: #f5f7fa;
      }

      .main-title {
        text-align: center;
        padding-top: 25px;
        padding-bottom: 10px;
      }

      .main-title h1 {
        font-size: 42px;
        font-weight: 700;
        color: #1f3c88;
        margin-bottom: 8px;
      }

      .main-title p {
        font-size: 20px;
        color: #5c677d;
      }

      .control-panel {
        background-color: white;
        padding: 20px;
        border-radius: 15px;
        box-shadow: 0px 2px 10px rgba(0,0,0,0.08);
      }

      .plot-panel {
        background-color: white;
        padding: 15px;
        border-radius: 15px;
        box-shadow: 0px 2px 10px rgba(0,0,0,0.08);
      }

    ")
  )
)
```

```
.btn-primary {
  background-color: #1f3c88;
  border-color: #1f3c88;
}

.btn-primary:hover {
  background-color: #163172;
  border-color: #163172;
}

"))
),

div(
  class = "main-title",

  h1("Exploración interactiva del dataset iris"),

  p("Visualización dinámica y análisis interactivo de datos con Shiny")
),

fluidRow(

  column(
    width = 3,

    div(
      class = "control-panel",

      h3("Controles"),

      tags$hr(),

      selectInput(
        inputId = "xvar",
        label = "Variable eje X:",
        choices = names(iris)[1:4],
        selected = "Sepal.Length"
      ),

      selectInput(
        inputId = "yvar",
        label = "Variable eje Y:",
        choices = names(iris)[1:4],
        selected = "Petal.Length"
      ),
    )
  )
)
```

```
selectInput(  
  inputId = "species",  
  label = "Filtrar especie:",  
  choices = c(  
    "Todas",  
    "setosa",  
    "versicolor",  
    "virginica"  
  ),  
  selected = "Todas"  
)  
  
checkboxInput(  
  inputId = "regresion",  
  label = "Mostrar línea de regresión",  
  value = TRUE  
)  
  
br(),  
  
downloadButton(  
  outputId = "downloadData",  
  label = "Descargar datos CSV",  
  class = "btn-primary btn-block"  
)  
)  
)  
  
column(  
  width = 9,  
  
  div(  
    class = "plot-panel",  
  
    tabsetPanel(  
  
      tabPanel(  
        "Gráfico",  
  
        br(),  
  
        plotOutput(  
          "scatterplot",  
          height = "550px"  
        )  
      ),  
  
      tabPanel(  

```

```
      "Resumen estadístico",  
  
      br(),  
  
      tableOutput("summary_table")  
    ),  
  
    tabPanel(  
      "Datos filtrados",  
  
      br(),  
  
      DTOutput("tabla_datos")  
    )  
  )  
)  
)  
)  
)  
)
```

b)

La aplicación incorpora distintos controles interactivos que permiten modificar dinámicamente la visualización y el análisis de los datos.

Los selectores `xvar` e `yvar` permiten elegir las variables numéricas que se representarán en los ejes X e Y del gráfico de dispersión.

El selector `species` permite filtrar las observaciones según la especie seleccionada, pudiendo visualizar todas las observaciones o únicamente una especie concreta.

Asimismo, la casilla `checkboxInput()` permite decidir si se desea añadir una línea de regresión sobre el gráfico.

Además, la aplicación incorpora:

- un diseño responsive,
- navegación por pestañas,
- tablas interactivas,
- descarga dinámica de datos filtrados,
- y una interfaz visual más moderna y profesional.

c)

A continuación, se implementa la lógica del servidor mediante la función `server()`. En esta parte se definen los elementos reactivos de la aplicación, el filtrado dinámico de datos y la generación automática de gráficos y tablas.

```

# Lógica del servidor
server <- function(input, output) {

  datos_filtrados <- reactive({

    if (input$species == "Todas") {
      iris
    } else {
      iris %>%
        filter(Species == input$species)
    }
  })

  output$scatterplot <- renderPlot({

    p <- ggplot(
      datos_filtrados(),
      aes_string(
        x = input$xvar,
        y = input$yvar,
        color = "Species"
      )
    ) +
    geom_point(
      size = 4,
      alpha = 0.85
    ) +
    theme_minimal(base_size = 15) +
    labs(
      title = paste(
        input$yvar,
        "vs",
        input$xvar
      ),
      subtitle = "Dataset iris",
      x = input$xvar,
      y = input$yvar,
      color = "Especie"
    ) +
    theme(
      plot.title = element_text(
        size = 24,
        face = "bold",
        color = "#1f3c88"
      ),
      plot.subtitle = element_text(
        size = 14,

```

```
      color = "#5c677d"
    ),
    legend.position = "bottom",
    panel.grid.minor = element_blank()
  )

  if (input$regresion) {
    p <- p +
      geom_smooth(
        method = "lm",
        se = FALSE,
        linewidth = 1.2
      )
  }

  p
})

output$summary_table <- renderTable({
  summary(datos_filtrados())
})

output$tabla_datos <- renderDT({
  datatable(
    datos_filtrados(),
    options = list(
      pageLength = 10,
      autoWidth = TRUE
    )
  )
})

output$downloadData <- downloadHandler(
  filename = function() {
    paste0(
      "datos_filtrados_",
      Sys.Date(),
      ".csv"
    )
  },
  content = function(file) {
```



```

    write.csv(
      datos_filtrados(),
      file,
      row.names = FALSE
    )
  }

)

}

```

Finalmente, la aplicación completa se ejecuta mediante:

```
shinyApp(ui = ui, server = server)
```

La aplicación desarrollada permite representar gráficamente distintas variables del dataset `iris`, aplicar filtros dinámicos según la especie seleccionada, visualizar resúmenes estadísticos y descargar los datos filtrados en formato CSV.

Además, el gráfico incorpora títulos dinámicos, estilos personalizados y una interfaz moderna que mejora considerablemente la experiencia de usuario.

d)

Dentro de una aplicación Shiny, el componente `ui` (*user interface*) se encarga de definir la interfaz gráfica que visualizará el usuario, incluyendo botones, selectores, gráficos y tablas.

Por su parte, el componente `server` contiene toda la lógica de la aplicación, es decir, las instrucciones necesarias para procesar datos y generar resultados dinámicos en función de las acciones realizadas por el usuario.

Finalmente, el concepto `reactive` permite crear objetos reactivos que se actualizan automáticamente cuando cambian los valores de entrada de la aplicación. Esto constituye uno de los elementos fundamentales de Shiny, ya que permite que gráficos, tablas y resultados se modifiquen en tiempo real de manera automática e interactiva.

Resultados de la aplicación

A continuación, se muestran distintas capturas de la aplicación desarrollada, donde se puede observar la interfaz principal, el gráfico interactivo, el resumen estadístico y la tabla dinámica de datos filtrados.

La aplicación final combina análisis estadístico, visualización interactiva y diseño moderno en una única interfaz desarrollada con **Shiny**. Gracias al uso de elementos reactivos, el usuario puede explorar los datos de forma dinámica e intuitiva, facilitando tanto el análisis exploratorio como la interpretación visual de la información.

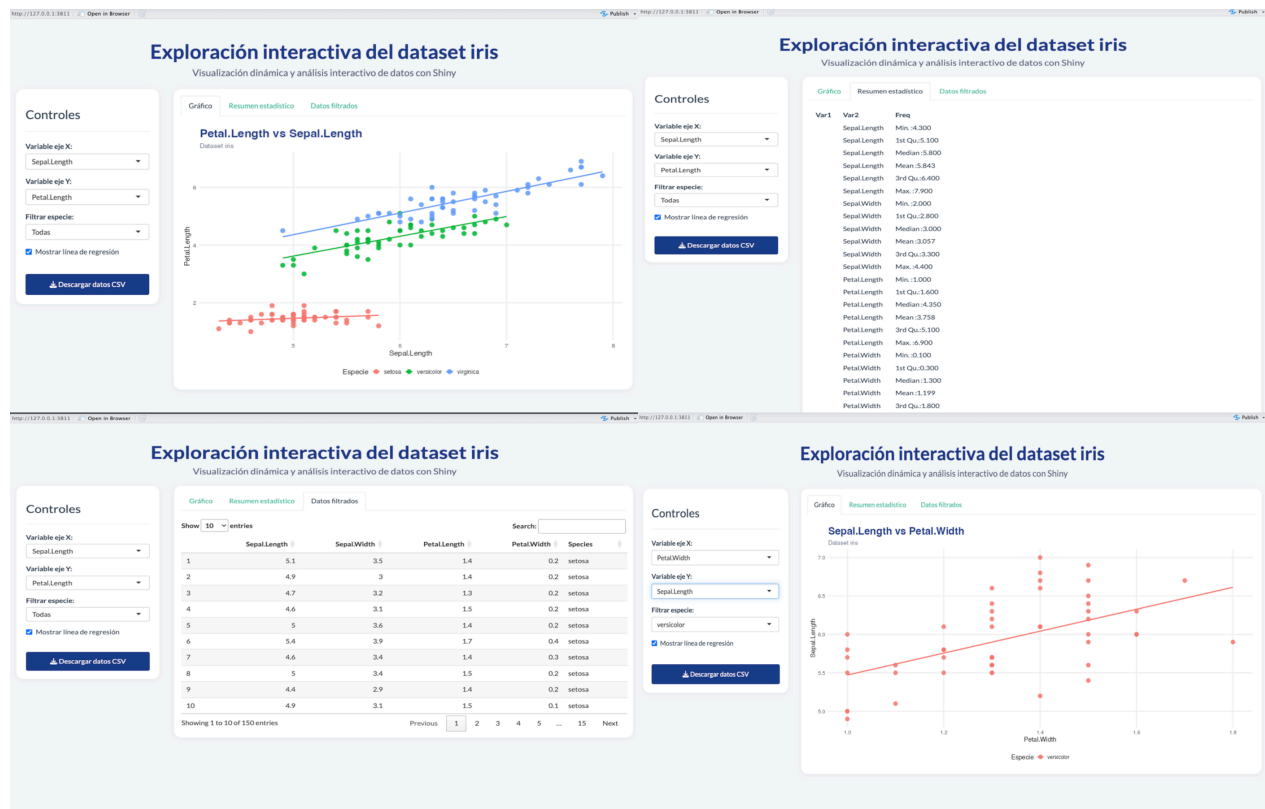


Figura 1. Capturas de la aplicación Shiny desarrollada para la exploración interactiva del dataset iris. Se muestran la interfaz principal, el gráfico dinámico, el resumen estadístico y la tabla interactiva de datos filtrados.

3.2. Ejercicio 4. Ética en aplicaciones Shiny

3.2.1. Enunciado

En el desarrollo de una aplicación interactiva con Shiny que permite explorar datos biomédicos y generar visualizaciones y resultados estadísticos en tiempo real, el diseño de la herramienta puede influir directamente en la interpretación que realiza el usuario.

Elaborar una respuesta argumentada en la que analice las responsabilidades éticas del desarrollador en relación con la interpretación de los resultados generados por la aplicación.

3.2.2. Resolución

El desarrollo de aplicaciones interactivas para el análisis de datos biomédicos conlleva una responsabilidad importante por parte del desarrollador, especialmente cuando los resultados pueden influir en decisiones clínicas o científicas. Aunque herramientas como Shiny facilitan mucho la visualización y exploración de datos, una aplicación mal diseñada puede llevar a interpretaciones equivocadas o a conclusiones poco fiables. Por ello, es fundamental que la información se presente de forma clara y fácil de interpretar. Aspectos como etiquetar correctamente los gráficos, evitar visualizaciones confusas o proporcionar contexto suficiente para entender los resultados son esenciales para que el usuario pueda interpretar adecuadamente el análisis realizado.

Además, una aplicación interactiva no debería transmitir la idea de que los resultados obtenidos son automáticamente válidos desde un punto de vista clínico sin supervisión profesional. También es importante ser transparente sobre las limitaciones de los modelos utilizados, la calidad de los datos y los posibles sesgos presentes en el análisis. Por ejemplo, si los datos empleados proceden mayoritariamente de un grupo concreto de población, los resultados podrían no ser igual de fiables para otros perfiles de pacientes, generando interpretaciones poco representativas. Del mismo modo, resulta recomendable que la aplicación incluya advertencias indicando que los resultados obtenidos tienen un carácter exploratorio o estadístico y que no sustituyen la valoración de un profesional sanitario.

Otro aspecto clave es la privacidad de los datos biomédicos. En aplicaciones reales, el desarrollador debe garantizar el cumplimiento de las normativas de protección de datos y evitar la exposición de información sensible de pacientes o participantes. Asimismo, también es recomendable mantener cierta trazabilidad de las acciones realizadas dentro de la aplicación, por ejemplo registrando los filtros aplicados o los parámetros seleccionados durante el análisis, ya que esto mejora la transparencia y facilita la reproducibilidad de los resultados obtenidos.

En definitiva, desarrollar aplicaciones con Shiny no consiste únicamente en que funcionen correctamente desde el punto de vista técnico, sino también en diseñar herramientas responsables, transparentes y orientadas a reducir posibles errores de interpretación o usos inadecuados de los resultados.

4. Sección 3: Bioconductor y análisis genómico

4.1. Ejercicio 5. Análisis diferencial de expresión génica con DESeq2

4.1.1. Enunciado

En este ejercicio se pide simular un conjunto de datos de expresión génica y llevar a cabo un análisis diferencial básico con el paquete DESeq2.

Se pide:

- Generar una matriz de contajes de expresión génica con una distribución de Poisson, con 1000 genes y 20 muestras. Hay que indicar claramente cómo se ha construido la matriz y qué estructura tienen filas y las columnas.
- Defina un vector o tabla de metadatos que diferencie los dos grupos experimentales (control y tratamiento) y cree el objeto DESeqDataSet corresponden.
- Ejecute el análisis diferencial con DESeq2 y obtenga la tabla de resultados. Hay que mostrar, como mínimo, las principales variables del resultado y explicar brevemente qué representan.
- Represente un histograma de los p-values y un gráfico de dispersión de $\log_2\text{FoldChange}$ versus $-\log_{10}(\text{pvalue})$. A continuación, intérprete brevemente los resultados obtenidos y comente qué permite observar cada uno de los gráficos.

4.1.2. Resolución

En este ejercicio se realiza una simulación sencilla de datos de expresión génica y posteriormente se lleva a cabo un análisis diferencial utilizando el paquete DESeq2, una de las herramientas más utilizadas dentro de Bioconductor para el análisis de datos de RNA-seq.

El objetivo principal es generar una matriz de contajes simulada, definir dos grupos experimentales y comprobar si el análisis es capaz de detectar genes diferencialmente expresados entre las condiciones de control y tratamiento.

a)

En primer lugar, se cargan las librerías necesarias para el análisis, concretamente DESeq2 y ggplot2:

```
# Cargar librerías necesarias
library(DESeq2)
library(ggplot2)
```

A continuación, se genera una matriz de contajes simulada utilizando una distribución de Poisson. En este caso se simulan 1000 genes y 20 muestras experimentales. Cada valor de la matriz representa el número de lecturas observadas para un gen concreto en una determinada muestra. Aunque el enunciado únicamente solicita generar una matriz de contajes simulada, se introduce de forma artificial una señal diferencial en un subconjunto de genes con el objetivo de que el análisis posterior con DESeq2 permita detectar diferencias de expresión más realistas e interpretables entre condiciones experimentales.

Para garantizar que los resultados puedan reproducirse posteriormente, se fija una semilla aleatoria mediante `set.seed()`:

```
# Generar matriz de contejes simulada
set.seed(123)

n_genes <- 1000
n_samples <- 20

conteos <- matrix(
  rpois(n_genes * n_samples, lambda = 50),
  nrow = n_genes,
  ncol = n_samples
)

# Introducir señal diferencial en 50 genes
genes_de <- 1:50

conteos[genes_de, 11:20] <- matrix(
  rpois(length(genes_de) * 10, lambda = 100),
  nrow = length(genes_de),
  ncol = 10
)
```

A continuación, se asignan nombres a las filas y columnas de la matriz. Las filas representan genes simulados y las columnas corresponden a las muestras experimentales:

```
# Asignar nombres a filas y columnas
rownames(conteos) <- paste0("Gene_", 1:n_genes)

colnames(conteos) <- paste0("Sample_", 1:n_samples)
```

Finalmente, se visualiza la estructura general de la matriz de contejes:

```
# Visualizar la estructura de la matriz de contejes
dim(conteos)
```

```
## [1] 1000 20
```

```
head(conteos[, 1:6])
```

```
##      Sample_1 Sample_2 Sample_3 Sample_4 Sample_5 Sample_6
## Gene_1      46      42      49      45      46      54
## Gene_2      58      65      54      43      44      44
## Gene_3      38      45      48      47      46      57
```

## Gene_4	50	43	53	46	48	52
## Gene_5	62	44	38	43	60	36
## Gene_6	53	37	42	49	59	59

La matriz obtenida contiene 1000 genes y 20 muestras experimentales. Los valores almacenados representan contejes enteros generados mediante una distribución de Poisson y permiten simular datos similares a los que podrían encontrarse en un experimento real de RNA-seq.

b)

Una vez generada la matriz de contejes, se define la información experimental asociada a las muestras mediante una tabla de metadatos (`colData`).

En este caso, las 20 muestras se dividen en dos grupos experimentales: un grupo control y un grupo tratamiento, con 10 muestras en cada uno.

```
# Definir vector de condiciones para cada muestra
condicion <- factor(
  c(
    rep("Control", 10),
    rep("Tratamiento", 10)
  )
)

metadata <- data.frame(
  row.names = colnames(conteos),
  condicion = condicion
)

metadata
```

```
##          condicion
## Sample_1    Control
## Sample_2    Control
## Sample_3    Control
## Sample_4    Control
## Sample_5    Control
## Sample_6    Control
## Sample_7    Control
## Sample_8    Control
## Sample_9    Control
## Sample_10   Control
## Sample_11 Tratamiento
## Sample_12 Tratamiento
## Sample_13 Tratamiento
## Sample_14 Tratamiento
## Sample_15 Tratamiento
## Sample_16 Tratamiento
## Sample_17 Tratamiento
```

```
## Sample_18 Tratamiento
## Sample_19 Tratamiento
## Sample_20 Tratamiento
```

A continuación, se crea el objeto `DESeqDataSet`, que constituye la estructura principal utilizada por `DESeq2` para almacenar tanto los contejos como el diseño experimental del análisis.

```
# Crear objeto DESeqDataSet
dds <- DESeqDataSetFromMatrix(
  countData = conteos,
  colData = metadata,
  design = ~ condicion
)

dds
```

```
## class: DESeqDataSet
## dim: 1000 20
## metadata(1): version
## assays(1): counts
## rownames(1000): Gene_1 Gene_2 ... Gene_999 Gene_1000
## rowData names(0):
## colnames(20): Sample_1 Sample_2 ... Sample_19 Sample_20
## colData names(1): condicion
```

El argumento `design = ~ condicion` indica que el análisis diferencial evaluará las diferencias de expresión génica entre las muestras control y las muestras tratamiento.

c)

Una vez creado el objeto `DESeqDataSet`, se ejecuta el análisis diferencial mediante la función `DESeq()`:

```
# Ejecutar análisis diferencial
dds <- DESeq(dds)
```

Posteriormente, se obtiene la tabla de resultados utilizando la función `results()`:

```
# Obtener tabla de resultados
resultados <- results(dds)

head(resultados)
```

```
## log2 fold change (MLE): condicion Tratamiento vs Control
## Wald test p-value: condicion Tratamiento vs Control
## DataFrame with 6 rows and 6 columns
```

##	baseMean	log2FoldChange	lfcSE	stat	pvalue	padj
##	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
## Gene_1	70.5806	1.027133	0.0859660	11.9481	6.64091e-33	1.17419e-31
## Gene_2	69.8507	0.995272	0.0973511	10.2235	1.55571e-24	1.54557e-23
## Gene_3	71.4798	1.017344	0.0849281	11.9789	4.58431e-33	9.10883e-32
## Gene_4	74.7318	1.050283	0.0835416	12.5720	3.01129e-36	1.05859e-34
## Gene_5	72.7453	1.064502	0.0848258	12.5493	4.01175e-36	1.30955e-34
## Gene_6	75.4796	1.053861	0.0827181	12.7404	3.52606e-37	1.61141e-35

La tabla generada contiene distintas métricas estadísticas calculadas para cada gen. La variable **baseMean** representa la expresión media normalizada del gen considerando todas las muestras. Por su parte, **log2FoldChange** indica el cambio de expresión entre tratamiento y control en escala log2, permitiendo identificar genes sobreexpresados o infraexpresados. También aparecen el error estándar (**lfcSE**), el estadístico del contraste (**stat**), el valor p (**pvalue**) y el valor p ajustado (**padj**) tras aplicar corrección por comparaciones múltiples mediante FDR.

A continuación, se ordenan los resultados según el valor ajustado y se calcula el número de genes significativos:

```
# Ordenar resultados por significación estadística
resultados_ordenados <- resultados[order(resultados$padj), ]

head(resultados_ordenados)
```

```
## log2 fold change (MLE): condicion Tratamiento vs Control
## Wald test p-value: condicion Tratamiento vs Control
## DataFrame with 6 rows and 6 columns
##      baseMean log2FoldChange    lfcSE      stat      pvalue      padj
##      <numeric>      <numeric> <numeric> <numeric> <numeric> <numeric>
## Gene_7      73.6914      1.11872 0.0842238   13.2827 2.91508e-40 4.44064e-38
## Gene_32     75.5738      1.10907 0.0834155   13.2958 2.44889e-40 4.44064e-38
## Gene_47     74.5331      1.11553 0.0836553   13.3348 1.45186e-40 4.44064e-38
## Gene_26     74.9758      1.07929 0.0834449   12.9341 2.88800e-38 3.29954e-36
## Gene_15     74.5390      1.07555 0.0835060   12.8799 5.84053e-38 4.44854e-36
## Gene_43     73.1451      1.08505 0.0841700   12.8912 5.04609e-38 4.44854e-36
```

```
# Número de genes significativos con FDR < 0.05
sum(resultados$padj < 0.05, na.rm = TRUE)
```

```
## [1] 52
```

Aunque inicialmente se introdujeron diferencias artificiales en 50 genes, el análisis detecta 52 genes significativos. Esta pequeña discrepancia puede deberse a la variabilidad aleatoria propia de la simulación y al comportamiento estadístico del procedimiento de contraste múltiple aplicado por DESeq2.

Para obtener una visión global de los resultados se utiliza también la función **summary()**:


```
# Resumen estadístico de los resultados
summary(resultados)
```

```
##
## out of 1000 with nonzero total read count
## adjusted p-value < 0.1
## LFC > 0 (up)      : 51, 5.1%
## LFC < 0 (down)    : 4, 0.4%
## outliers [1]      : 0, 0%
## low counts [2]    : 543, 54%
## (mean count < 50)
## [1] see 'cooksCutoff' argument of ?results
## [2] see 'independentFiltering' argument of ?results
```

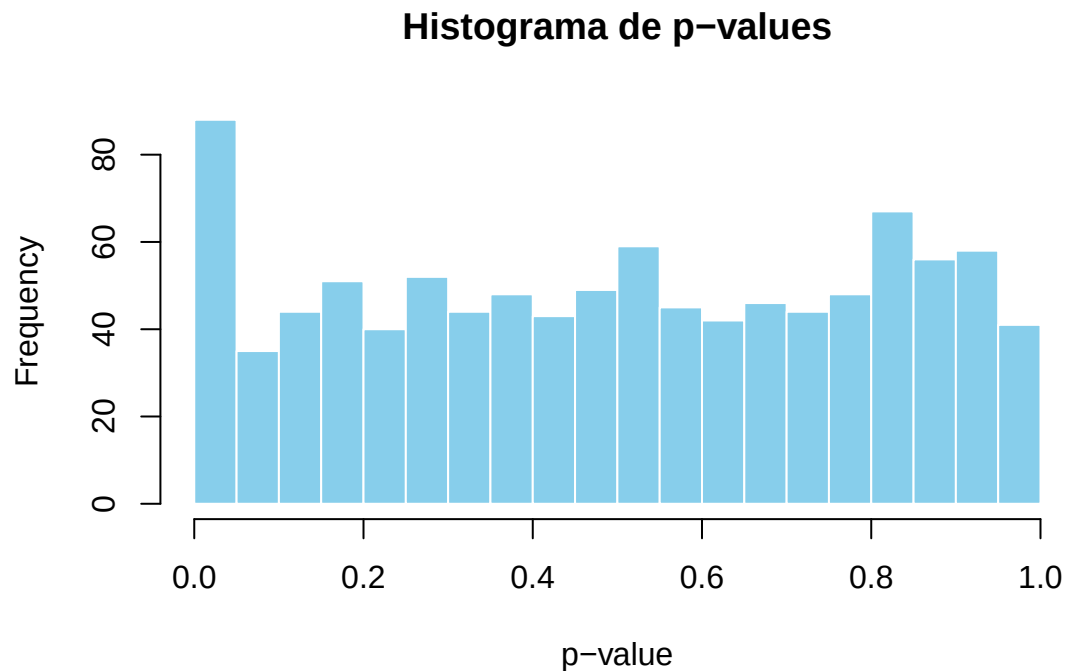
En este caso, dado que se introdujeron genes diferencialmente expresados de forma artificial, es esperable detectar un subconjunto de genes significativos asociados al grupo tratamiento. Esto permite comprobar que el análisis realizado por DESeq2 funciona correctamente y es capaz de identificar diferencias de expresión entre condiciones experimentales.

d)

A continuación, se representan dos gráficos habituales en análisis diferencial de expresión génica.

Histograma de p-values

```
# Histograma de p-values
hist(
  resultados$pvalue,
  breaks = 30,
  col = "skyblue",
  border = "white",
  main = "Histograma de p-values",
  xlab = "p-value"
)
```



En el histograma puede observarse una acumulación de valores p próximos a cero. Esto indica que existen genes con diferencias estadísticamente significativas entre los grupos control y tratamiento. El resultado es coherente con la simulación realizada, ya que previamente se introdujeron genes con mayor expresión en el grupo tratamiento.

En un análisis real de RNA-seq, este tipo de distribución suele interpretarse como evidencia de expresión diferencial en una parte de los genes analizados.

Volcano plot: log2FoldChange vs -log10(pvalue)

```
# Preparar datos para volcano plot
df_res <- as.data.frame(resultados)

df_res$gen <- rownames(df_res)

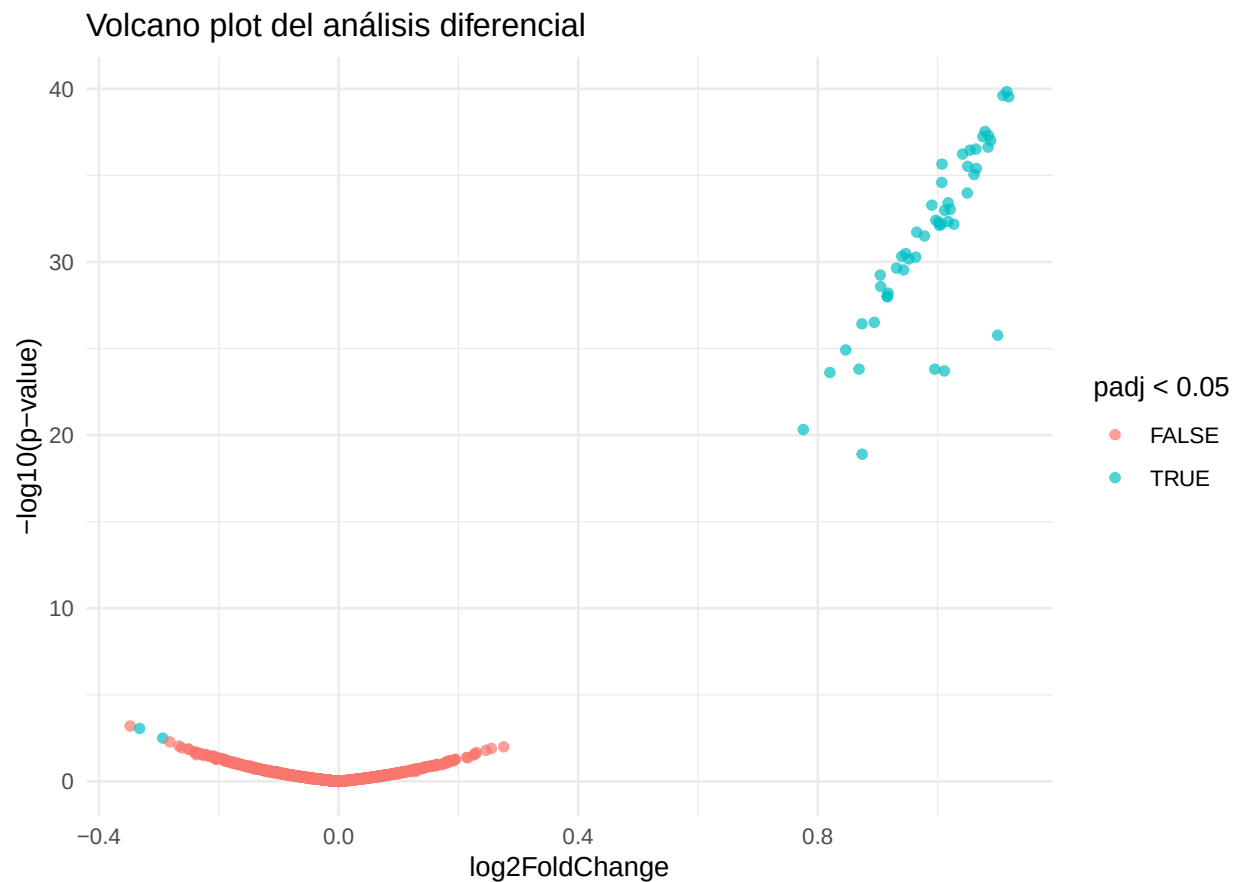
df_res$significativo <- ifelse(is.na(df_res$padj), FALSE, df_res$padj < 0.05)

# Volcano plot
ggplot(
  df_res,
  aes(
    x = log2FoldChange,
    y = -log10(pvalue)
  )
) +
  geom_point(
    aes(color = significativo),
  )
```

```

alpha = 0.7
) +
theme_minimal() +
labs(
  title = "Volcano plot del análisis diferencial",
  x = "log2FoldChange",
  y = "-log10(p-value)",
  color = "padj < 0.05"
)

```



Este gráfico permite visualizar simultáneamente el tamaño del cambio de expresión y el nivel de significación estadística obtenido para cada gen. En el eje horizontal se representa el $\log_2\text{FoldChange}$, que indica cuánto cambia la expresión de un gen entre tratamiento y control. En el eje vertical se representa $-\log_{10}(\text{pvalue})$, por lo que los genes situados en posiciones más altas presentan mayor significación estadística. Los genes más alejados del centro horizontal corresponden a aquellos que muestran cambios de expresión más elevados. Además, los genes situados en la parte superior del gráfico presentan valores p más pequeños y, por tanto, mayor evidencia estadística de expresión diferencial.

En este caso puede observarse un subconjunto de genes destacados, coherente con la simulación realizada previamente. El volcano plot resulta especialmente útil porque permite identificar de forma visual genes candidatos relevantes que combinan simultáneamente un cambio de expresión elevado y una alta significación estadística.